MACAU UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Computer Science and Engineering

Faculty of Innovation Engineering

Final Year Project Proposal

Title: AI-Assisted Teaching System for Programming Courses — Backend & Database Focus

Student Name : Liu Shiyu

Student No. : 1220032841

Supervisor : Tian Jinyu

October, 2025

Abstract

Programming courses increasingly enroll larger cohorts while maintaining limited teaching assistant capacity, leading to delayed, inconsistent, or shallow feedback on student code. This project proposes a back-end–centric AI-assisted teaching system that evaluates student submissions, assigns grades, and produces constrained, curriculum-aligned improvement suggestions. The system integrates program analysis and sandboxed execution with a retrieval-augmented generation (RAG) pipeline targeting course-specific materials (syllabus, lecture notes, starter code, rubric exemplars). A concept-aware constraint layer ensures guidance never references topics students have not yet learned (e.g., pointers) and enforces safety, academic integrity, and privacy. A principled grading rubric quantifies correctness, robustness, code quality, efficiency, documentation, test quality, and integrity signals; the rubric is encoded in the database and applied uniformly by evaluators—human or AI. The back-end provides RESTful APIs for submission, evaluation, feedback retrieval, rubric/version management, analytics, and identity-scoped history. A relational database persists submissions, versions, grades, feedback, rubric criteria, curriculum concepts, RAG artifacts, and execution logs. The methodology includes static/dynamic analysis, unit tests, mutation testing signals, plagiarism similarity checks, and LLM-based formative feedback grounded by course materials through RAG. Evaluation compares AI grades and feedback against instructor/TA judgments using accuracy error (MAE/MAPE), inter-rater agreement (Cohen's κ), and learning outcome gains on a controlled pilot. The proposal details architecture, algorithms, schema, security and fairness considerations, required software/hardware, a six-month Gantt plan, and references.

Table of Contents

Abstract 1
1. Introduction 3
2. Objectives 4
3. Related work 5
4. Methodology 6
5. Required hardware and software 11
6. Project planning 13
References 17

1. Introduction

Over the past decade, programming education has faced two compounding pressures: rising enrollment and increasing language/tool diversity in introductory and intermediate courses. Instructors aim to provide high-frequency, formative feedback—yet manual grading is time-consuming, uneven across graders, and often delayed. Traditional autograders mitigate the problem by executing unit tests against student programs, but their feedback is binary, narrow to functional correctness, and rarely explanatory. Recent advances in large language models (LLMs) for code understanding and generation (e.g., Chen et al., 2021; Rozière et al., 2023) enable richer diagnostics, rationale, and actionable suggestions. However, naively applying LLMs in education introduces risks: hallucinations, suggestions that outpace the syllabus (e.g., using pointers or advanced libraries), grading inconsistency, and privacy/compliance constraints.

This project addresses those gaps by combining established software-engineering signals (compilation status, unit tests, coverage, mutation analysis, static warnings, cyclomatic complexity) with an LLM feedback loop that is retrieval-augmented with course-specific artifacts (Lewis et al., 2020). Retrieval-augmented generation (RAG) grounds the model's feedback in the course's definitions, coding standards, and canonical examples, and it provides traceable citations within the feedback. A concept ontology aligned with the course schedule drives concept-aware constraints so the system never recommends techniques or APIs students have not yet learned. A robust data schema supports accurate record-keeping (for auditing and appeal), reproducibility (versioning of prompts, rubric, test suites), and analytics (e.g., common error clusters, concept mastery trajectories).

The student benefit is faster, higher-quality, and level-appropriate feedback; the teaching benefit is reliable, scalable grading with transparent rubrics and artifact-backed rationales. The project focuses on the back-end and database, delivering the core evaluation service and persistence layer that course websites or LMS front-ends can call.

2. Objectives

1. Design and implement a back-end service that ingests code submissions, orchestrates sandboxed execution and analyses, invokes an LLM with RAG over course artifacts, and returns both a numeric grade and constrained, curriculum-aligned feedback within a defined SLA.

2. Develop a principled, configurable grading rubric that aggregates functional correctness, robustness, code quality, efficiency, documentation, test quality, and integrity signals into a final score; encode it in the database for versioned, auditable application.

3. Build a concept-aware constraint mechanism that enforces student-level and assignment-level topic boundaries (e.g., disallow pointers before Week N) across both automated grading logic and LLM feedback.

4. Implement a RAG pipeline that indexes course materials (syllabi, lecture slides, exemplars, style guides) with chunking and embeddings, retrieves relevant passages per submission, and conditions the LLM to generate grounded, citation-linked explanations.

5. Design a secure, auditable database that records submissions, versions, grades, tests, feedback, RAG citations, prompts, model versions, rubric versions, and student concept mastery states; support longitudinal analytics and appeals.

6. Establish reliability, fairness, and integrity controls, including execution sandboxes, rate limits, plagiarism detection signals, sensitive data protection, and bias/consistency evaluation of grading.

7. Empirically evaluate grading accuracy and feedback helpfulness by comparing to instructor/TA assessments on held-out assignments, reporting inter-rater agreement, score error, ablations (with/without RAG, with/without concept constraints), and student outcome proxies

3. Related work

Autograding for programming. Traditional autograders compile/run student submissions against instructor tests to assign correctness scores. Surveys highlight their adoption and limitations (Ihantola et al., 2010): functional tests do not capture design quality, code readability, or minor conceptual misunderstandings. Mutation testing (Jia & Harman, 2011) increases robustness by measuring whether tests detect injected faults; coverage metrics further illuminate test thoroughness.

LLMs for code understanding and feedback. Code-capable LLMs have demonstrated competence on benchmarks like HumanEval (Chen et al., 2021). Models such as CodeBERT (Feng et al., 2020), CodeT5 (Wang et al., 2021), and Code Llama (Rozière et al., 2023) support code representation and generation. In an educational context, LLMs can explain compiler errors, propose refactors, and suggest tests. Yet they may hallucinate or propose out-of-scope constructs, motivating curriculum-aware constraints.

Retrieval-Augmented Generation (RAG). RAG mitigates hallucinations by conditioning generation on retrieved passages (Lewis et al., 2020). In educational use, retrieval from course artifacts aligns feedback with local definitions and style rules while enabling citations, improving transparency and trust.

Academic integrity. Similarity detection tools (e.g., JPlag; Prechelt et al., 2002) help detect collusion and reuse. Policy-aware LLMs require guardrails so feedback does not divulge complete solutions or introduce prohibited techniques.

Rubrics and educational measurement. Rubric-based assessment supports consistent grading and formative feedback (Brookhart, 2013). Inter-rater agreement (Cohen's κ, 1960) is a standard reliability measure; we apply it to compare AI-assisted grading against human graders.

4. Methodology

The proposed system follows a back-end–centric pipeline that couples conventional program analysis and sandboxed execution with retrieval-augmented, curriculum-aware language-model feedback. At a high level, a student's submission (comprising source files, language metadata, and an assignment identifier) enters through an authenticated API gateway and is queued for processing. A containerized executor builds and runs the program against instructor-provided tests under strict resource limits, while collectors record functional outcomes, code coverage, execution time, memory use, and relevant logs. In parallel, language-specific static analyzers compute complexity, duplication, and style metrics and surface compiler or linter warnings. These signals are normalized into a submission profile that summarizes what worked, what failed, and where potential design or stylistic issues may lie. Rather than allowing a language model to improvise explanations, the system constructs a retrieval query from this profile—blending assignment keywords, failing test names, error messages, and the assignment's learning objectives—and uses it to fetch semantically relevant passages from a vector index built over course artifacts (syllabus, slides, rubrics, exemplar solutions where permitted, and style guides). The resulting evidence bundle, together with a compact code digest or AST summary, conditions the model to generate feedback that is both traceable and aligned with local course definitions.

A central design element is the concept-aware constraint mechanism. The course's curriculum is represented as a directed acyclic graph of concepts (for example, variables, conditionals, loops, functions, arrays, pointers, dynamic memory, recursion, abstract data types, and object-oriented patterns), each tagged with the week it is introduced and, where useful, pre-requisites. Each assignment declares a set of allowed concepts, and each student has a mastery state that can advance as the term proceeds. The constraint mechanism influences generation in two ways. First, it instructs the model, at prompt time, to stay within the allowed subset by naming disallowed notions explicitly and by describing permitted constructs positively; this reduces the chance that suggestions leap ahead of the syllabus. Second, a post-generation validator scans the feedback—both text and any suggested snippets—using token patterns and lightweight parsing to detect out-of-scope APIs or idioms (for instance, pointer syntax or heap allocation calls before pointers or dynamic memory have been taught). If violations are detected, the system either redacts the offending portions or regenerates the feedback with stricter guidance. The net effect is that a student who has not learned pointers receives suggestions framed in terms of arrays and indices, whereas a later assignment can legitimately discuss address-taking and manual memory management.

Grading blends measurable software-engineering signals with rubric-guided qualitative judgments, but the latter are bounded tightly by the former to ensure consistency. Functional correctness is treated as the anchor criterion and contributes the largest portion of the grade: roughly forty percent arises from the proportion of tests passed (including hidden, edge-case-

focused tests) and the absence of runtime exceptions or timeouts. Robustness contributes about ten percent and is estimated from mutation testing where feasible, from defensive input checks, and from the stability of behavior under slight input perturbations. Code quality and style account for approximately fifteen percent and are characterized by the density of linter warnings per thousand lines of code, naming and formatting conformance to the course style guide, and duplication metrics. Algorithmic efficiency contributes around ten percent and is judged by empirical scaling on input tiers relative to targets discussed in lectures or assignment sheets. Documentation and clarity add another ten percent, reflecting the presence of purposeful headers, docstrings, and brief invariants. When the course requires students to submit tests, test quality contributes an additional ten percent and emphasizes coverage and the ability to detect faults. Finally, about five percent is reserved for academic integrity signals; the system computes normalized code fingerprints and similarity measures to flag potential collusion or reuse. While these percentages can be tuned per course or assignment, the initial values produce a balanced aggregation that privileges correctness yet leaves space to reward good engineering practice.

Operationally, the grader proceeds without resorting to rigid checklists. The execution results, coverage figures, static metrics, and performance counters form a vector of signals that map naturally to rubric descriptors. The model is then asked to provide criterion-level rationales and a proposed grade band, but its latitude is limited: the final numeric score is computed from the signals and rubric weights, and any adjustment based on narrative justification is capped to a narrow window to accommodate borderline cases where, for example, all tests pass but coverage reveals shallow testing or where a single minor style violation should not unduly penalize an otherwise excellent submission. The feedback emphasizes formative guidance: it identifies the most probable root causes for failing tests, points to specific passages in the retrieved course materials, and suggests concrete, within-scope steps to improve; notably, it avoids supplying full solutions or introducing concepts the student has not yet studied.

The retrieval pipeline itself is intentionally conservative. Course documents are segmented into semantically coherent chunks with moderate overlap to preserve context across slide bullets or paragraph boundaries. Embeddings are computed using sentence-level or code-aware models and stored either in a dedicated vector database or in PostgreSQL via pgvector, depending on deployment constraints. At run-time, the retriever scores candidate passages not only by vector similarity but also by lightweight heuristics tied to the submission profile—for instance, passages that mention specific failing test names or error symbols receive a small boost. Deduplication and source diversity filters avoid returning near-identical fragments from the same document, which helps keep prompts compact and encourages the model to cite varied evidence. Each feedback paragraph includes bracketed citation tags that map to the retrieved chunks, enabling instructors to audit whether the system truly grounded its claims.

The persistence layer is designed to support auditability, reproducibility, and longitudinal analytics. Rather than enumerating tables as a checklist, it is useful to view the schema as circles

of provenance around a submission. At the core lies the submission itself—files, language, and metadata—tied to an assignment and a student, each of which, in turn, belongs to a course and term. Surrounding the submission are execution runs that capture sandbox images, timestamps, statuses, and canonical URIs to logs and artifacts. Results from tests, static analysis, and performance measurements live adjacent to the run to preserve time-aligned context. The rubric—versioned independently so that courses can refine descriptions without corrupting prior grades—defines criteria, weights, and level descriptors. Each computed score references the rubric version used, records the numeric outcome and band, and stores a concise justification string that anchors the number to concrete evidence (for example, the percentage of hidden tests passed or the measured time at the largest input tier). Feedback items are stored at the criterion level and carry serialized citations to the retrieved passages. The concept ontology, concept sets for assignments, and per-student mastery states reside in a separate but linked cluster of tables so that the same course ontology can be reused across multiple offerings. The retrieval index is represented by document entries and their chunks; every retrieval event, including the rank and score of each returned chunk, is logged so that researchers can later study which pieces of material were most influential. Finally, prompt templates, model configurations, and complete generation traces—including prompt and output snapshots—are stored so that any grade can be reconstructed precisely even after models or prompts evolve.

From the perspective of integrators, the back-end exposes a small, stable set of endpoints that encapsulate these behaviors without forcing clients to manage internal details. A submission endpoint accepts code and returns an identifier that callers can poll or subscribe to for status updates. A retrieval endpoint presents the assembled summary of signals—tests, coverage, static metrics, and performance—associated with a submission, while another returns the final grade and the feedback with human-readable citations. Course staff can access endpoints that describe the rubric for a given assignment, request re-evaluation under a newer rubric or prompt version when pedagogically justified, reindex updated course materials after lecture revisions, or modify concept sets as the schedule changes. All endpoints require role-appropriate authentication, and student identifiers are scoped per course to reduce the surface area of personally identifiable information.

Security and integrity measures are interwoven rather than bolted on. Each build and test runs within a non-privileged container hardened with seccomp or AppArmor profiles, network access is disabled by default, and CPU, memory, and wall-clock limits prevent abuse or accidental denial of service from pathological programs. File systems are mounted read-only except for a designated workspace that is destroyed after execution, and artifact URIs are pre-signed and short-lived. Logs are structured with correlation identifiers that tie together queue entries, executor runs, retrieval calls, and model generations, which makes incident response and debugging tractable. Privacy is protected by encrypting selected columns—such as student emails or names—at rest and by minimizing the presence of PII in prompts; the code digest and AST summaries are anonymized and only include what is necessary for analysis. To maintain

fairness, the team plans regular calibration exercises that compare AI-assisted grades against instructor or TA grades on stratified samples; deviations beyond acceptable thresholds trigger rubric or prompt adjustments. Integrity tooling computes code similarity and fingerprints across the cohort and prior semesters, but any penalties are imposed only after human review; the system's role is to inform, not to adjudicate unilaterally.

Evaluation emphasizes not only accuracy but also educational value. On two or more assignments, a stratified sample of submissions will be dual-graded by human staff, and the system's scores will be compared using mean absolute error for numeric alignment and Cohen's kappa for agreement across grade bands. To understand whether retrieval and constraints matter, ablation studies will disable grounding or concept filters and observe the impact on both agreement and the incidence of out-of-scope suggestions. Teaching assistants—blinded to the feedback's origin—will rate helpfulness on a short Likert scale, and student learning gains will be approximated by the improvement between initial and resubmitted versions whenever the course allows resubmission. Latency and throughput will be monitored during peak deadlines to verify that the queueing and executor layers scale appropriately and that the end-to-end experience meets reasonable service levels. Error analyses will cluster failures by language, assignment type, and recurrent misconception patterns, which, in turn, will inform adjustments to the retrieval index and to future lecture materials.

In sum, the methodology integrates three strands—rigorous automated testing and analysis, retrieval-grounded and curriculum-aware language-model coaching, and meticulous provenance in the data layer—to deliver grading that is fast, consistent, auditable, and educationally aligned. The approach treats the model as a disciplined component in a measured pipeline rather than an oracle, and it makes room for iteration by capturing the evidence and decisions needed to refine rubrics, prompts, and constraints over time.

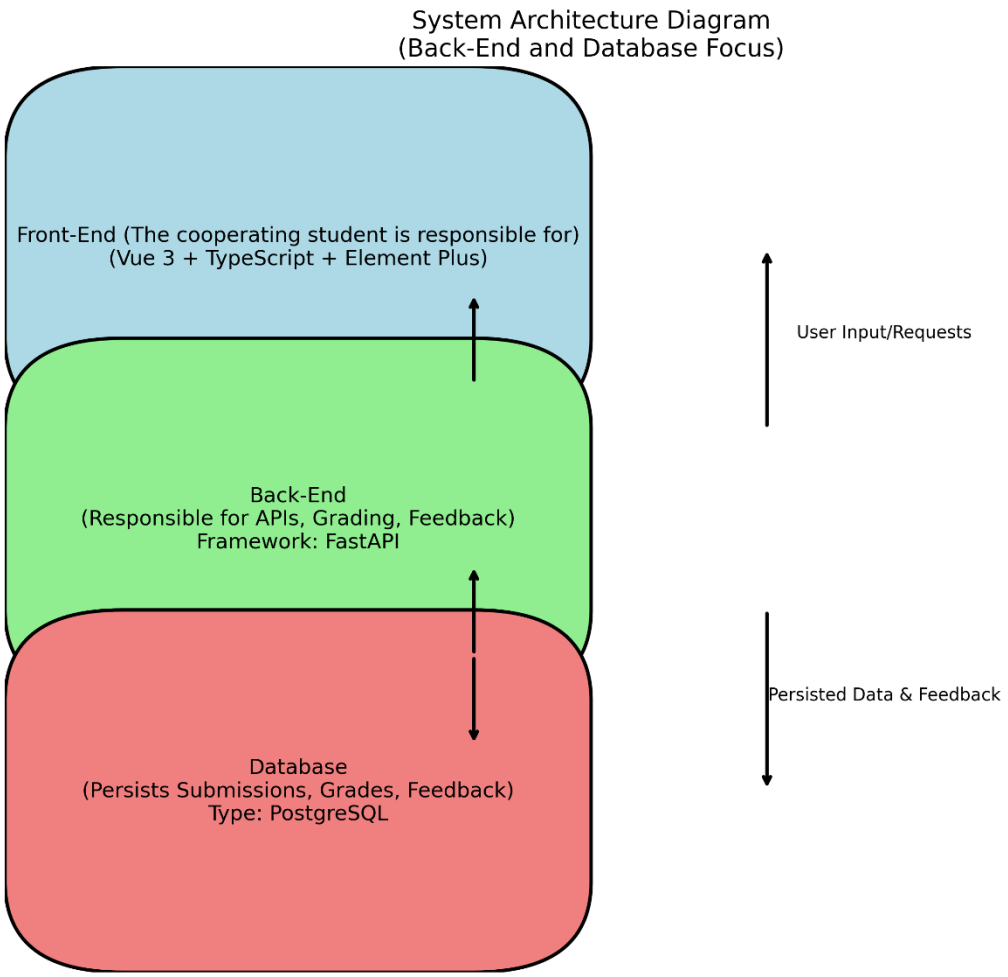

Fig. 1 System Framework

5. Required hardware and software

The platform is designed to operate reliably on modest institutional infrastructure while leaving headroom for expansion. For a course of typical size, a single application server with eight to sixteen virtual CPUs and thirty-two to sixty-four gigabytes of memory is sufficient to process submissions in parallel, provided that a separate worker pool handles build and execution tasks. Disk performance matters more than raw capacity during deadlines: sustained read and write rates on the order of a hundred to two hundred megabytes per second ensure that sandboxes can be created and torn down quickly and that logs and artifacts can be persisted without bottlenecks. Network bandwidth requirements are moderate because executors run without external access and because most artifacts are internal; nevertheless, the system benefits from a fast internal network when multiple workers share an artifact store. A GPU is not necessary for the core grading workflow; embedding computation for the retrieval index can be performed offline during ingestion or on a smaller accelerator if available, and, should the team later experiment with local model inference, a single modern GPU can accelerate that path without being essential to the initial deployment.

On the software side, the back-end is implemented in Python 3.11 using a modern asynchronous web framework to expose RESTful APIs and to simplify concurrency. FastAPI, served by an ASGI server such as Uvicorn, offers type-hinted routes and automatic documentation that facilitate integration with learning-management systems. Background work is dispatched through a lightweight queue—Celery or RQ backed by Redis are both suitable—so that submissions can be processed concurrently and so that transient failures can be retried without blocking clients. Containerized execution relies on Docker or containerd with carefully crafted images per supported language, each pinned to known compiler or interpreter versions and equipped with the course-approved testing frameworks. Resource controls and seccomp or AppArmor profiles are applied at the container level to enforce the sandbox's security invariants. Static analyzers are selected per language—pylint or flake8 for Python, clang-tidy and cpplint for C or C++, and Checkstyle for Java—and are configured to reflect the course style guide rather than generic defaults.

Persistent data lives in PostgreSQL, chosen for its transactional guarantees, mature tooling, and extensions that suit this application. The schema's evolution is managed by migrations so that new terms can introduce concepts or rubric refinements without jeopardizing historical records. When the retrieval index is colocated with the transactional store, the pgvector extension provides efficient similarity search over embeddings; in larger deployments, a dedicated vector store such as FAISS may be preferable, but both options are viable. Sensitive columns use pgcrypto to encrypt at rest, and role-based access control within the database limits exposure. Observability relies on a Prometheus and Grafana stack to collect and visualize metrics across the API, workers, and database; structured logs include correlation identifiers so that individual

submissions can be traced from API ingestion through execution and feedback generation. The model integration layer is abstracted behind a configuration registry that records provider, model name, and decoding parameters per task; this abstraction makes it possible to swap models or providers as institutional policy or cost constraints evolve. Finally, the retrieval subsystem includes document ingestion tools to convert slides and PDFs to text, normalize formatting, and chunk content into embedding-ready segments; embeddings are computed with sentence-level or code-aware models depending on the corpus, and the ingestion process can be scheduled to run automatically when instructors update materials.

Taken together, these hardware and software choices favor reliability, auditability, and maintainability over novelty. They provide a secure execution environment for untrusted code, a durable and queryable record of grading decisions, and a disciplined interface between automated signals and model-generated feedback. Most importantly, they align with the educational goals of timely, consistent, and level-appropriate guidance by making the system's behavior both configurable—through rubrics, concept sets, and prompts—and transparent—through logs, citations, and reproducible traces.

6. Project planning

The project is planned over a six-month window, organized as a sequence of tightly coupled phases that move from requirements capture and architectural design to implementation, pilot evaluation, and delivery. The plan deliberately frontloads clarification of scope—languages, assignment archetypes, rubric criteria, and the concept ontology—so that later work on execution sandboxes, analysis pipelines, and the retrieval index can build on stable definitions. Once the data model and infrastructure are in place, we implement the execution layer for compiling and testing student programs, together with static metrics and performance counters; these form the quantitative substrate on which the grading rubric operates. In parallel, we prepare the retrieval system by ingesting course artifacts, producing embeddings, and validating that queries derived from typical error profiles recover the intended passages. Only after these components are reliable do we integrate the grading and coaching model, which is prompted with rubric descriptors, constrained by the concept graph, and grounded by retrieved evidence. Integrity checks and policy enforcement are then layered in to surface similarity signals and to ensure that feedback stays within the allowed topic set. The final implementation phase exposes instructor-facing endpoints for rubric maintenance, re-indexing of updated instructional materials, and analytics, which enable staff to monitor common misconceptions and calibrate grading. The pilot period evaluates the end-to-end system on real assignments, compares AI-assisted grades against instructor or TA judgments, and collects latency and throughput data under deadline conditions. The last weeks are reserved for hardening—security review, performance tuning, and log retention configuration—and for the preparation of the final report and demonstration package.

The ordering of phases reflects critical dependencies. Database schema decisions and message contracts between the API, worker queue, and sandbox determine how submissions, signals, and feedback are persisted and retrieved; changing these late would ripple across every component. Likewise, the retrieval index depends on document normalization and chunking choices that should be validated before prompt engineering begins, because the format and granularity of citations influence the structure of feedback. The concept-aware constraint layer relies on the course's concept ontology and the per-assignment allowed-concept sets; these must be settled in time to validate post-generation filtering and to seed the prompt templates with accurate prohibitions. By contrast, integrity checks such as fingerprint-based similarity detection can be integrated later without destabilizing the core pipeline, as their outputs are advisory to instructors and have no bearing on the immediate scoring logic. The plan includes modest slack between integration checkpoints to absorb inevitable iteration on prompts, retrieval heuristics, and rubric descriptors discovered during the pilot. Each phase culminates in tangible acceptance artifacts: for design, a reviewed API specification and threat model; for infrastructure, a migration-backed schema and a reproducible local development environment; for execution, passing test batteries across languages; for retrieval, measured top-k accuracy on synthetic and historical queries; for

grading, agreement metrics on a stratified sample; and for the pilot, a brief evaluation report and issue backlog prioritized for hardening.

Resourcing emphasizes reliability and auditability. The back-end and database lead is responsible for schema evolution, migration discipline, and the durability of decision traces; the same role owns the API surface, job orchestration, and sandbox templates that guarantee consistent execution. Collaboration with the teaching staff is embedded at the boundaries between phases, particularly when setting rubric weights and level descriptors, curating documents for the retrieval index, and defining the allowed-concept sets per assignment. Risk is managed through early spikes in areas most likely to derail schedules—sandbox isolation policies, OCR quality for slide decks, and the robustness of static analyzers across student codebases. Where uncertainty remains high, the plan includes fallback modes, such as a signal-only grader that can operate during model service outages and a minimal retrieval mode that defaults to rubric passages when assignment documents are incomplete. The evaluation period is timed to coincide with two consecutive assignments in the term so that we can measure learning gains between first and second submissions when resubmission is permitted. Throughout, observability is treated as a first-class deliverable: correlation identifiers tie submissions to executor runs, retrieval events, and model generations; these are critical both for debugging during the pilot and for producing the final, evidence-based report.

The following GANTT chart visualizes the schedule, durations, and dependencies. Dates are illustrative for a six-month project starting in early November 2025; in actual deployment, the start date can be shifted to align with the course calendar without altering dependencies or

sequencing.

Fig. 2 Project GANTT Chart

In practice, this chart functions as both a planning and a communication artifact. By encoding dependencies explicitly—for example, placing prompt and reconciliation work after the retrieval index has been validated and positioning integrity checks after constraints are proven to filter out-of-scope suggestions—the chart helps the team avoid rework and localizes risk. The exclusion of weekends sets realistic expectations for throughput and developer availability, and the adjacency of pilot assignments enables side-by-side analysis of agreement with human

graders while the prompts and retrieval heuristics are still fresh in mind. Should early phases complete ahead of schedule, additional time can be invested in expanding language adapters or deepening analytics; if delays occur, the schedule preserves a fallback path that still delivers a signal-only baseline with auditable scoring and minimal feedback, ensuring continuity for the course while the full retrieval-grounded and concept-aware layer is finalized.

References

[1] Andrade, H. G. (2005). Teaching with rubrics: The good, the bad, and the ugly. *College teaching*, *53*(1), 27-31.

[2] Brookhart, S. M. (2013). *How to create and use rubrics for formative assessment and grading*. Ascd.

[3] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., ... & Zaremba, W. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

[4] Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and psychological measurement*, *20*(1), 37-46.

[5] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., ... & Zhou, M. (2020). Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.

[6] Ihantola, P., Ahoniemi, T., Karavirta, V., & Seppälä, O. (2010, October). Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli calling international conference on computing education research* (pp. 86-93).

[7] Jia, Y., & Harman, M. (2010). An analysis and survey of the development of mutation testing. *IEEE transactions on software engineering*, *37*(5), 649-678.

[8] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, *7*(3), 535-547.

[9] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, *33*, 9459-9474.

[10] Prechelt, L., Malpohl, G., & Philippsen, M. (2002). Finding plagiarisms among a set of programs with JPlag. *J. Univers. Comput. Sci.*, *8*(11), 1016.

[11] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., ... & Ma, S. (2020). Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*.

[12] Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., ... & Synnaeve, G. (2023). Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

[13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

[14] Wang, Y., Wang, W., Joty, S., & Hoi, S. C. (2021). Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859*.